

ChessDotNet 1.0.0 – Comprehensive Documentation

Introduction to ChessDotNet

ChessDotNet is an open-source chess library for .NET (written in C#) that provides the core rules and mechanics of chess, allowing developers to build chess games and applications without implementing the game logic from scratch. Its primary purpose is to handle move generation and validation, enforce rules (including special moves and endgame conditions), and track game state. Key features of ChessDotNet include robust move validation (supporting special moves like castling and *en passant*), check/checkmate and stalemate detection, and FEN string parsing/generation for saving or initializing game states ¹. The library even supports several popular chess variants out of the box – such as **Atomic Chess**, **King of the Hill**, **Three-Check Chess**, **Antichess** (losing chess), **Horde**, and **Racing Kings** – with the rules for these variants handled internally ² ³. It also implements rule-based conditions like draw by the fifty-move rule (a draw can be claimed after 50 half-moves without pawn moves or captures) ³. (Note: other draw conditions like threefold repetition or insufficient material were planned but not implemented in version 1.0.0 ⁴.)

In summary, ChessDotNet offers a clean and simple API for chess logic. Developers can use it to manipulate chess pieces and board positions programmatically, validate moves (prevent illegal moves), enumerate all possible moves, detect game termination (checkmate/stalemate), and more ⁵. It is a **rules engine** for chess (not an AI engine), making it easier for both beginners and experts to develop chess-based software without worrying about the intricacies of move legality and game rules.

Installation

ChessDotNet is distributed as a NuGet package, making it straightforward to install into your project. It targets **.NET Standard 1.3**, which means it is compatible with a wide range of platforms (including .NET Framework 4.6+ and all modern .NET Core/5+ runtime environments) ⁶. There are two ways to install ChessDotNet:

- **NuGet Package Manager Console:** In Visual Studio, open the Package Manager Console and run:

```
Install-Package ChessDotNet -Version 1.0.0
```

- **.NET CLI:** Using the .NET CLI, you can add the package by executing:

```
dotnet add package ChessDotNet --version 1.0.0
```

These commands will download and add the ChessDotNet 1.0.0 library to your project, along with any dependency it might have (ChessDotNet itself has no additional package dependencies).

Optional – Chess Variants: If you intend to use the extended chess variants support, you should also install the companion package **ChessDotNet.Variants** (which provides classes for Atomic Chess, Horde, etc.) ⁷. For example, use `Install-Package ChessDotNet.Variants -Version 1.0.0` in the Package Manager Console to add the variants library. (The base ChessDotNet package covers standard chess rules; the `ChessDotNet.Variants` package provides the variant-specific rule extensions.)

After installation, you can import the library in your C# code with the appropriate namespace (likely `ChessDotNet`) and begin using its classes.

Basic Usage Examples

Once installed, ChessDotNet enables you to start a chess game, make moves, and query the game state with just a few API calls. Below are some basic examples demonstrating common usage:

- **Starting a new game:** You can initialize a standard chess game by creating a `ChessGame` object. By default, a new `ChessGame` starts with the standard initial board setup and White to move:

```
using ChessDotNet;  
  
ChessGame game = new ChessGame(); // start a new standard chess game
```

This creates a game with the pieces in their starting positions.

- **Making a move:** To make a move, create a `Move` instance specifying the source square, destination square, and the player making the move (White or Black). Then call `ChessGame.MakeMove()` to apply the move to the game:

```
Move whiteMove = new Move("e2", "e4", Player.White);  
bool moveOk = game.MakeMove(whiteMove, true);
```

In this example, White moves a pawn from e2 to e4. The `MakeMove` method returns a boolean indicating if the move was successful/legal. The second parameter (`true`) tells the game to actually apply the move if it is valid. After a successful move, the game's internal state updates – the pawn moves to the new square, and it becomes Black's turn.

- **Checking game status:** The library provides ways to inspect the game's current status. For example, you can check whose turn it is via the `game.WhoseTurn` property (returns a `Player` enum value, either `Player.White` or `Player.Black`). After each move, you might also want to see if the move resulted in check or game end:

```
Console.WriteLine("Turn: " + game.WhoseTurn); // e.g. "Turn: Black"  
Console.WriteLine("FEN: " +  
game.GetFen()); // current board in FEN notation  
Console.WriteLine("Game result: " + game.GameResult); // e.g.  
"None" (ongoing) or "Draw", "WhiteWins", etc.
```

The `GetFen()` method returns a FEN string representing the current board state, which is useful for debugging or saving the game. The `GameResult` property (of type `GameResult` enum) tells if the game is ongoing or if one side has won or a draw occurred. For an ongoing game, this might be a value indicating "None" or "InProgress" (depending on implementation). If White delivered checkmate, for example, `GameResult` would be set to White's victory status.

Putting it together, here's a simple usage demonstration: start a game, make a couple of moves, and print out the state:

```
ChessGame game = new ChessGame();
game.MakeMove(new Move("e2", "e4", Player.White), true); // White pawn from
e2 to e4
game.MakeMove(new Move("e7", "e5", Player.Black), true); // Black pawn from
e7 to e5
Console.WriteLine(game.GetFen()); // prints FEN after two moves
if (game.GameResult != GameResult.Ongoing) {
    Console.WriteLine("Game over: " + game.GameResult);
}
```

This will output the FEN string of the position after the moves, and if the game had ended (e.g., via checkmate or draw), it would indicate the result.

Full API Reference

Below is an overview of the main public classes, enums, and members provided by the ChessDotNet library. (For brevity, only the most important members are listed. All classes and members are in the `ChessDotNet` namespace unless otherwise noted.)

- **ChessGame class:** Represents a chess game (standard or variant). This is the central class managing game state.
- **Constructors:**
 - `ChessGame()` – Initializes a new game with the standard initial chess setup (pieces in starting positions, White to move).
 - `ChessGame(string fen)` – Initializes a game from a given FEN string. Allows starting from arbitrary positions (the FEN must include side to move, castling rights, etc., as usual).
- **Properties:**
 - `Player WhoseTurn` – Indicates which player's turn it is (`Player.White` or `Player.Black`).
 - `GameResult GameResult` – Indicates the outcome of the game. Possible values include "Ongoing" (or "None") if the game is not yet finished, `WhiteWins` (if White has won by checkmate or opponent resignation), `BlackWins`, or `Draw`. This property updates when the game ends (checkmate, stalemate, draw, etc.).
 - `List<Move> AllMoves` – A list of all moves that have been made in the game so far, in chronological order. Each move in this list contains details such as its SAN (Standard Algebraic Notation).
 - (*Board representation*) – `ChessGame` internally tracks the board configuration, but it does not expose a direct `Board` matrix. You can use methods like `GetPieceAt(square)` (if available) or generate a FEN string via `GetFen()` to examine the board. The FEN string

encodes piece positions, so `GetFen()` is the primary way to retrieve the full board state easily.

• Methods:

- `bool MakeMove(Move move, bool commit)` - Attempts to make the specified move. If `commit` is `true`, the move will be applied to the game state if it is legal. If `commit` is `false`, the method can be used to *validate* a move without altering state. The return value is `true` if the move is legal (and in committed mode, applied). Illegal moves will return `false` (and not change the game state). This method automatically handles special moves (castling, en passant, pawn promotion) according to the rules.
- `IEnumerable<Move> GetValidMoves(Player player)` - Generates all legal moves for the given player from the current position ⁸. This can be used, for example, to highlight possible moves or to implement an AI. The list will be empty if the player has no legal moves.
- `bool HasAnyValidMoves(Player player)` - Returns `true` if the given player has at least one legal move available, or `false` if no moves are possible ⁸. This is a quick way to check for checkmate or stalemate: if a player has no moves and their king is in check, it's checkmate; if no moves and not in check, it's stalemate.
- `bool IsInCheck(Player player)` - (If provided) Checks whether the specified player's king is currently in check. This might be exposed via a property or method. If not directly exposed, you can infer check by whether certain moves are legal or by using the `GameResult` when no moves are available.
- `string GetFen()` - Returns the FEN (Forsyth-Edwards Notation) representation of the current game state. This string includes the board layout, side to move, castling rights, en passant target square if any, halfmove clock, and fullmove number. Use this to serialize or save the position, or to debug the game state.
- `ChessGame Clone()` - (If provided) Creates a deep copy of the game state, allowing you to experiment with moves without affecting the original game. (Alternatively, you can create a new `ChessGame` from the FEN of the current game to simulate moves on a copy.)

- **Move class:** Represents a chess move from one square to another (and optional promotion).

• Constructors:

- `Move(string fromSquare, string toSquare, Player player)` - Creates a move where a piece belonging to `player` moves from the coordinate `fromSquare` (e.g., "e2") to `toSquare` (e.g., "e4"). Coordinate strings use algebraic notation (file + rank).
- `Move(string fromSquare, string toSquare, Player player, PieceType promotion)` - Creates a pawn promotion move, where `promotion` specifies the piece type to promote to (e.g., Queen, Rook, etc.). Use this when moving a pawn to the last rank.

• Properties:

- `string OriginalPosition` - The starting square of the move (e.g., "e2").
- `string NewPosition` - The destination square of the move (e.g., "e4").
- `Player Player` - Which player is making the move (White or Black).
- `PieceType PromotionPiece` - If this move is a pawn promotion, this indicates the piece type the pawn is promoted to (e.g., `PieceType.Queen`). Otherwise it may be `PieceType.None` or null for non-promotion moves.
- `string SAN` - The Standard Algebraic Notation representation of the move (e.g., "e4", "Nf3", "O-O" for king-side castle, etc.). This is typically populated after the

move is applied in a game (since disambiguation and check/checkmate notation are determined in context). You can retrieve a move's SAN from the `AllMoves` history after it's made ⁹.

- `bool IsCapture` – (If exposed) A flag indicating if the move involves a capture. This might be deducible from context (e.g., SAN containing "x").
- **Methods:** (Usually `Move` is a simple data class; it may not have many methods beyond perhaps `ToString()`.)
 - `override string ToString()` – Returns a human-readable description of the move, possibly in SAN or long algebraic form (e.g., "White: e2->e4"). (The exact output depends on implementation.)
- **Player enum:** An enumeration indicating a player/color. Possible values: `Player.White` and `Player.Black`. This is used to specify which side is acting or being queried. For example, `game.WhooseTurn` returns a `Player` value, and the `Move` constructor requires a `Player` to ensure the move is being made by the correct side. (There is no concept of a "neutral" player in standard chess rules, so the enum is essentially two-valued.)
- **PieceType enum (or class):** The library defines the types of chess pieces. If using an enum, it likely includes values such as `None` (empty square), `Pawn`, `Knight`, `Bishop`, `Rook`, `Queen`, `King`. In ChessDotNet, piece information is primarily handled internally or via FEN characters, but promotion moves require specifying a piece type. The `PieceType` (or a similar construct) is used to denote that (e.g., promoting a pawn to `PieceType.Rook`). The library might also expose piece information through methods or FEN interpretation, but generally you will deal with piece types only when needed (like promotions).
- **GameResult enum:** Indicates the result of the game. Possible values (based on typical usage) could be:
 - `GameResult.Ongoing` (or `None`) – The game is not yet finished (this is the default state during play).
 - `GameResult.WhiteWins` – White has won (usually by checkmate; ChessDotNet will set this when Black is checkmated or otherwise loses).
 - `GameResult.BlackWins` – Black has won.
 - `GameResult.Draw` – The game ended in a draw. Draws can occur by stalemate, the fifty-move rule, or other agreed/claimed draw conditions. (Note: in version 1.0.0, draws by threefold repetition or insufficient material are not auto-detected, so these would not set `Draw` without external handling.)

You can check this property after each move to determine if the game is over and what the outcome is.

- **Chess variant classes:** (In the `ChessDotNet.Variants` namespace, provided by the optional *variants package*.) For each supported chess variant, there is a class that inherits from `ChessGame` and modifies the rules:
- `AtomicChessGame` – Implements **Atomic Chess** rules. In Atomic Chess, **any capture causes an "explosion"**: the capturing piece, the captured piece, and **all pieces (of either color) on immediately adjacent squares** are removed from the board as well (kings are immune to exploding, i.e., a king is not removed by an adjacent explosion, but kings cannot castle through or into explosion danger). The goal is still to checkmate the opponent's king ¹⁰, but because of explosions, checkmate patterns differ from standard chess.

- `KingOfTheHillChessGame` – Implements **King of the Hill** (KOTH) rules. In KOTH, a player can win by moving their king to the center of the board. **If a king occupies any of the four central squares (d4, d5, e4, e5), that player immediately wins the game** ¹¹. Normal checkmate rules still apply as an alternate win condition, but the added victory condition makes for a different strategic emphasis.
- `ThreeCheckChessGame` – Implements **Three-Check Chess** rules. In this variant, **a player wins by delivering check three times** to the opponent ¹² (accumulated across the game). All normal chess rules apply (you can also win by checkmate, but often the triple-check happens first). The game state needs to track how many checks each side has delivered; the class handles this internally.
- `AntichessGame` – Implements **Antichess** rules (also known as **Losing Chess** or Giveaway chess). The objective in Antichess is to *lose* all your pieces. **Capturing is compulsory** whenever a capture move is available ¹³. The king has no special status (there is no check or checkmate – your king can be captured like any piece). A player wins by running out of pieces or by stalemating themselves (since stalemate in this variant means the stalemated player cannot move and thus has no remaining legal moves, which typically counts as a win for that player in losing chess). The `AntichessGame` class adjusts move generation to enforce mandatory captures and removes check rules.
- `HordeChessGame` – Implements **Horde Chess** rules. Horde is an asymmetrical variant where one side (White by default) has a “horde” of pawns instead of the usual pieces. Typically, **White has 36 pawns (a large formation covering ranks 1-4) and Black has the standard set of pieces** ¹⁴. White wins by checkmating the Black king (using its pawns as the attacking force), and Black wins by capturing all of White’s pawns (thus eliminating the horde) ¹⁵. The `HordeChessGame` class sets up the non-standard initial position and modifies win conditions accordingly.
- `RacingKingsChessGame` – Implements **Racing Kings** rules. In Racing Kings, both players race to get their king to the top rank (the 8th rank) as quickly as possible. **The ultimate goal is to move your king to the eighth rank before your opponent does** ¹⁶. This variant has a special starting position (kings begin on the bottom rank along with a selection of pieces, and there are no pawns). There is no concept of check or checkmate; the game is purely a race. The `RacingKingsChessGame` class sets up the board in the Racing Kings initial layout and checks for the win condition when a king reaches rank 8. (It likely disables check rules and treats any situation where one king gets to the end as a win immediately.)

Using a variant class is similar to using `ChessGame`: you instantiate (for example) `AtomicChessGame game = new AtomicChessGame();` to start an Atomic Chess game, then use `game.MakeMove(...)`, etc. The variant classes override the necessary logic to enforce their special rules. All other usage (checking `WhoseTurn`, calling `GetValidMoves`, etc.) remains the same as in standard chess.

Advanced Usage Scenarios

Beyond the basics, ChessDotNet provides flexibility for more advanced use cases. Here we discuss a few scenarios: **custom rules and variants**, **move validation utilities**, and **state serialization**.

- **Custom Rules / Extending the Library:** If the built-in variants are not enough or you want to implement a custom chess rule, ChessDotNet’s design allows extension. The variant classes (like `AtomicChessGame`, etc.) are examples of extending the base `ChessGame` class to override certain behaviors. You could similarly create your own subclass of `ChessGame` to implement custom win conditions or movement rules. For instance, if you wanted a variant where a piece moves in an unconventional way, you could override the move-generation functions (though this

requires understanding of the library's internal design). Since the project is open-source, you can also modify the source directly or fork it under the MIT license to create new features. In general, the clean separation of concerns (move validation, game state, etc.) makes it feasible to adapt the rules. (If creating a custom variant, it may be easiest to start by copying one of the existing variant classes as a template.)

- **Move Validation and Game Logic Checks:** One common use of a chess library is to **validate moves** or to generate all legal moves from a position (for AI or UI move-highlighting). ChessDotNet provides the `GetValidMoves(player)` method which returns all possible legal moves for the given side in the current position ⁸. This is useful for implementing chess AI agents or ensuring user moves are legal in a UI. For example, you could call `var moves = game.GetValidMoves(game.WhoseTurn)` to get all moves available to the current player. To check if a specific move is legal **without making it**, you have a couple of options:
 - Use `MakeMove` with the `commit` flag set to false (if you want to leverage the library's internal validation). For example: `bool legal = game.MakeMove(potentialMove, false);` – if this returns true, the move is legal, but the game state remains unchanged.
 - Generate the list of valid moves via `GetValidMoves` and see if the move you're interested in appears in that list.

Additionally, the `HasAnyValidMoves(player)` method is a quick way to determine if a player is in a terminal situation ⁸. If `HasAnyValidMoves(currentPlayer)` is false, the game is over – you can then check `GameResult` to know if it was checkmate or stalemate. The library internally ensures moves that would put your own king in check are not considered valid, so you can trust that any move returned by `GetValidMoves` has already been vetted for legality.

- **Serialization (FEN, SAN, PGN):** ChessDotNet makes it easy to serialize and deserialize game states:
 - **FEN strings:** As mentioned, you can create a game from a FEN string and retrieve the current FEN via `GetFen()`. This is extremely useful for saving game progress or communicating state (for example, sending the position to a frontend or storing in a database). The FEN produced by `GetFen()` will include up-to-date halfmove and fullmove counters, etc., so you can resume the game exactly.
 - **SAN notation:** Every move made in a `ChessGame` is recorded and can be output in Standard Algebraic Notation. The `Move.SAN` property provides the SAN for that move ⁹. For example, after a move is made, you might do `Console.WriteLine(game.AllMoves.Last().SAN)` to display it. SAN notation will include special notation for captures (e.g., `Qxe5`), checks (`+`) and checkmates (`#`) as appropriate. This is useful for displaying moves to users or constructing a PGN.
 - **PGN construction:** While ChessDotNet does not appear to have a full PGN parser/writer in version 1.0.0 (PGN support was not listed among features), you can still build a PGN using the data it provides. By iterating through `game.AllMoves` and concatenating the SAN strings with move numbers, you can form a PGN record of the game. For example, you can number the moves and list SAN strings: `"1. e4 e5 2. Nf3 Nc6 3. ..."` and so on. (Ensure to handle move numbers and results according to PGN specification if needed.)
 - **Cloning and analysis:** If you want to analyze hypothetical scenarios (e.g., test a sequence of moves), you can clone the game or use FEN to create a duplicate game state. This way, you can make moves on the clone without affecting the original game. This technique is useful for AI search (exploring possible moves) or for providing an "undo" feature in a UI by rolling back to a previous FEN.

In summary, ChessDotNet's advanced capabilities let you adapt it to many scenarios: you can enforce or explore alternate rules, integrate it into a chess engine or AI by generating moves and simulating outcomes, and serialize game states for storage or communication. The library is designed to handle the heavy lifting of chess logic so you can focus on higher-level development.

Version Compatibility and Dependencies

Platform Compatibility: ChessDotNet 1.0.0 is built for **.NET Standard 1.3**, which ensures broad compatibility ⁶. In practice, this means it can be used on .NET Framework 4.6 and later, .NET Core 1.0 and later, .NET 5/6/7, Mono, Xamarin, and any other environment supporting at least .NET Standard 1.3. The NuGet Gallery confirms compatibility with numerous targets (including .NET 5, .NET 6, .NET 7, .NET 8, etc.) due to this .NET Standard base ¹⁷ ¹⁸ – so you should have no issues integrating ChessDotNet into modern .NET projects on Windows, Linux, or Mac.

Dependencies: The ChessDotNet library has **no third-party dependencies**. The NuGet package is self-contained, relying only on the core .NET libraries. This makes installation and deployment simple – you won't need to pull in large chess engines or additional components. (The size of the NuGet package is relatively small, around 37 KB, indicating the library is lightweight.)

ChessDotNet.Variants Compatibility: The optional `ChessDotNet.Variants` package (for variant game support) targets the same .NET Standard 1.3, so it has the same compatibility range as the core library. Ensure that if you use the variants assembly, your platform meets that requirement as well (which most will, given the above).

There are no known version conflicts or special instructions needed to use ChessDotNet 1.0.0 on newer frameworks – it should work out-of-the-box on any framework that implements .NET Standard 1.3 or higher. Always make sure to use the correct version of the package for your project (here, we focus on 1.0.0, which is the latest as of writing). If you upgrade to a higher version in the future (if one becomes available), check the release notes for any changes in target frameworks or breaking changes.

Internal Design and Architecture Overview

Understanding the internal design of ChessDotNet can help in extending the library or debugging. The architecture is relatively straightforward, focusing on object-oriented representations of chess concepts:

- **Core classes and modules:** At the heart of the library is the `ChessGame` class, which encapsulates the board state, the rules for move legality, and the logic for detecting special conditions like check or mate. Internally, `ChessGame` likely maintains an 8×8 representation of the board (for example, as a two-dimensional array or other matrix, where each entry might be a piece or empty). Each piece could be represented by a combination of a type and an owner (color) – for instance, an internal `Piece` struct or class with fields for `PieceType` and `Player`, or simply an encoded value (some libraries encode pieces as chars or integers). The library uses this board representation to generate and validate moves.
- **Move generation & validation:** The logic for generating moves for a given piece is implemented within `ChessGame` (or associated helper classes). For each piece type, ChessDotNet checks the board and determines which moves are legal:
- **Sliding pieces** (rook, bishop, queen) – iterate through squares in each direction until blocked.

- **Knight moves** – check up to 8 L-shaped destinations.
- **Pawn moves** – handle straight moves, double-step from initial position, captures (diagonals), and en passant captures (which are only legal immediately after an opponent's pawn moves two steps and lands adjacent).
- **King moves** – one square in any direction, plus castling logic (which requires checking that the path is clear and no squares are under attack).

After generating candidate moves, **validation** steps ensure that moves do not leave the player's own king in check (this involves a hypothetical move application and king safety check). The library's "move validation" feature means it will not allow a move that is illegal by chess rules ¹ – for example, moving a bishop like a knight, or moving into check.

- **Game state and turn management:** ChessDotNet tracks whose turn it is (`WhoseTurn`) and toggles it after a successful move. It also maintains the **halfmove counter** (for the 50-move draw rule) and **fullmove number** as part of the game state (these are reflected in FEN output). When `MakeMove` is called and a move is executed, internal counters and flags (like castling availability or en passant eligibility) are updated accordingly:
 - If a pawn moves two squares, an en passant target is recorded.
 - If a pawn move or capture occurs, the halfmove counter resets (otherwise it increments).
 - If a king or rook moves, castling rights for that side are revoked as needed.
 - If a move is a capture, the captured piece is removed from the board and the capturing piece moved to its square.
 - If a move is a promotion, the pawn is replaced by the new piece type.
- **Check, Checkmate, Stalemate detection:** The library likely provides methods or uses post-move checks to determine if the move caused check or checkmate. When you call `MakeMove`, after applying the move it may set a flag if the opposing king is now in check. Checkmate is determined if it's the opponent's turn and they have no legal moves *and* their king is in check. Stalemate is determined if the opponent has no legal moves *and* the king is **not** in check. ChessDotNet can identify these conditions automatically. In fact, as soon as a move leads to a position where the opponent has no moves, `GameResult` will be updated to the appropriate outcome (BlackWins, WhiteWins, or Draw for stalemate) which you can query ⁸.
- **Class hierarchy:** The design uses inheritance to support variants. The base `ChessGame` class contains all the standard chess logic. Each variant class (in the `ChessDotNet.Variants` assembly) inherits from `ChessGame` and overrides specific methods or properties to implement variant-specific behavior. For example, `AtomicChessGame` might override the method that handles captures to implement explosions, and override the victory condition check to account for kings being captured via explosion (or to handle that kings cannot be adjacent, depending on variant rules). `ThreeCheckChessGame` likely introduces a counter for checks and overrides the method that checks for game end, declaring a win if the counter reaches three. This inheritance structure makes it easy to maintain and extend rules without duplicating the entire chess logic – each variant class can call into the base class for most operations and then apply its custom rules at key points.
- **Modules and separation:** The repository is organized such that the core logic is in the `ChessDotNet` project (handling standard chess and providing the general framework), and an optional `ChessDotNet.Variants` project adds extended rules. There are also accompanying test projects (e.g., `ChessDotNet.Tests` and `ChessDotNet.Variants.Tests`) indicating a test-driven approach for verifying move legality and rule enforcement. This separation suggests

a clean architecture: one can use the core library independently if only standard chess is needed, or include the variants module for additional features.

- **Performance considerations:** ChessDotNet prioritizes correctness and clarity of the chess rules implementation. It may not use ultra-optimized bitboard techniques (common in high-performance chess engines) given its focus and the fact it's written in straightforward C#. Instead, it likely uses readable algorithms (loops over board squares, etc.). For the majority of use cases (GUI apps, casual engines, educational purposes), the performance is more than sufficient. The library handles typical move generation and validation in milliseconds, which is fine for real-time user interaction. If you plan to build a high-performance AI, you might need to incorporate additional optimizations or use ChessDotNet as a correctness check alongside more optimized move generation, but for most scenarios the provided performance is adequate.

Overall, the internal design of ChessDotNet is modular and maintainable: each chess concept (move, piece, game) is reflected in the code structure, and the use of object-oriented principles (encapsulation of game state, inheritance for variants) makes it easier to understand or extend. This architecture choice also means new chess variants or custom rules can be integrated with minimal changes, by leveraging the existing framework.

(For those interested in the source code structure: the GitHub repository shows directories for the core library and variants, and a test suite that covers move validation, special moves, and variant-specific scenarios. Reading the tests can provide additional insight into how the library expects each rule to behave.)

Licensing and Contribution

License: ChessDotNet is released under the **MIT License**, a permissive open-source license ¹⁹ ²⁰. This means you are free to use the library in personal or commercial projects, modify it, and redistribute it, provided you include the license notice. The MIT license imposes virtually no restrictions aside from attribution – it's very friendly for all kinds of use cases.

Contributing: The project was originally developed by Thomas Daniels (GitHub user **ProgramFOX**). As an open-source project, contributions were welcome via the typical GitHub workflow – forking the repository, making improvements, and submitting pull requests. You could also report issues or request features through the GitHub issues tracker. However, it's worth noting that the author has indicated the project is **no longer actively maintained** ²¹. This means that while you can still fork the project and modify it, you may not get official updates or responses on the original repository.

If you find a bug or want to extend ChessDotNet, you have a few options: - **Fork and fix:** Since MIT permits it, you can fork the repository and apply your own fixes or enhancements. Many community forks can exist; for instance, someone might create an updated version if needed (as of now, 1.0.0 is the latest). - **Contact the community:** You might find other developers who have used ChessDotNet (on forums or GitHub) if you need help or want to collaborate on improvements. - **Use as-is:** Of course, you can simply use ChessDotNet as provided. Despite being unmaintained, the library is stable for the features it offers, and for standard chess rules it should work reliably.

If you do contribute to a fork or your own project based on ChessDotNet, remember to retain the original license and credits.

Acknowledgments: The project was hosted on GitHub, and its NuGet package was published by the author. The inclusion of variant logic and a comprehensive test suite reflects the thoroughness of

contributions up to version 1.0.0. Even if official development has stopped, the existing codebase serves as a useful foundation for chess programming in C#.

Source Code and Further Links

For more information, you can refer to the following resources related to ChessDotNet:

- **Source Code Repository (GitHub):** The full source code for ChessDotNet is available on GitHub at <https://github.com/thomas-daniels/Chess.NET>. This repository contains the core library, the variants extension, sample usage code, and unit tests. Browsing the code can help you understand finer details of implementation, and the readme there includes a summary of features ²². (Note: the repository might be archived as read-only due to the maintenance status, but it is still accessible for reading and forking.)
- **NuGet Package Page:** The NuGet page for ChessDotNet provides installation instructions and shows the latest version number. You can visit <https://www.nuget.org/packages/ChessDotNet/1.0.0> for details. Similarly, the variants package is listed at <https://www.nuget.org/packages/ChessDotNet.Variants/1.0.0>. These pages confirm the .NET Standard compatibility and allow you to download the package or view its statistics.
- **Documentation & Examples:** (Aside from this compiled documentation) There isn't an official standalone documentation website for ChessDotNet, but the usage is straightforward. You may find community examples on forums or GitHub by searching for "ChessDotNet" usage. For instance, the **ChessDotNet.AI** project by another developer demonstrates using ChessDotNet to build simple chess AI agents ²³.
- **Chess Variants References:** If you plan to use the chess variants and want to understand their rules better, you might consult external resources:
 - Atomic Chess rules on Lichess ¹⁰ or Wikipedia.
 - King of the Hill rules (Chess.com forums or lichess) ¹¹.
 - Three-Check rules (Lichess or Chess.com) ¹².
 - Antichess/Losing Chess rules on Wikipedia ¹³.
 - Horde Chess rules (Chess.com variants section) ²⁴.
 - Racing Kings rules (Lichess or Chess.com) ¹⁶.

Understanding these will help you use the variant classes correctly.

In conclusion, ChessDotNet (1.0.0) is a powerful yet easy-to-use library for implementing chess games in C#. Its comprehensive rule enforcement and support for popular variants make it a versatile choice. With the information in this documentation, developers of all experience levels should be able to get started quickly – installing the library, making moves, and integrating chess logic into their applications. Happy coding, and enjoy building your chess program!

Sources: The information above was compiled from the ChessDotNet source repository and official pages, including the GitHub README (feature list and notes) ²⁵ ³, the NuGet gallery (installation instructions and compatibility) ²⁶ ⁶, and variant rule descriptions from chess community resources for accuracy on game rules ¹¹ ¹² ¹³ ¹⁶. For further details, please refer to the provided links.

- 1 2 3 4 7 19 20 21 22 25 **GitHub - thomas-daniels/Chess.NET: [No longer maintained] A .NET chess library, written in C#**
<https://github.com/thomas-daniels/Chess.NET>
- 5 8 23 **GitHub - danvanderboom/ChessDotNet.AI**
<https://github.com/danvanderboom/ChessDotNet.AI>
- 6 17 18 26 **NuGet Gallery | ChessDotNet 1.0.0**
<https://www.nuget.org/packages/ChessDotNet>
- 9 **GitHub - Irepaty/ChessDotNetCore**
<https://github.com/Irepaty/ChessDotNetCore>
- 10 **Atomic chess - Wikipedia**
https://en.wikipedia.org/wiki/Atomic_chess
- 11 **How to play King of the Hill? - Chess Forums**
<https://www.chess.com/forum/view/chess960-chess-variants/how-to-play-king-of-the-hill>
- 12 **Three-check • Check your opponent 3 times to win the game. - Lichess**
<https://lichess.org/variant/threeCheck>
- 13 **Losing chess - Wikipedia**
https://en.wikipedia.org/wiki/Losing_chess
- 14 24 **Horde Chess - Chess Variants**
<https://www.chess.com/terms/horde-chess>
- 15 **Destroy the horde to win! - Lichess**
<https://lichess.org/variant/horde>
- 16 **What is Racing Kings chess? | Chess.com Help Center**
<https://support.chess.com/en/articles/8614048-what-is-racing-kings-chess>